arXiv:2502.11537v4 [cs.LG] 10 May 2026

# Simulus: Combining Improvements in Sample-Efficient World Model Agents

**Lior Cohen** Technion
**Kaixin Wang** Microsoft Research
**Bingyi Kang** ByteDance Seed
**Uri Gadot** Technion
**Shie Mannor** Technion

## Abstract

World models (WMs) represent the frontier of sample-efficient reinforcement learning, but their complexity leaves many promising improvements unrealized due to the significant expertise and effort required to identify and integrate them. Inspired by Rainbow, which showed that individually known improvements to DQN complement each other and can be effectively combined, we take on this challenge and ask whether the same principle applies to world model agents. We introduce Simulus, a modular token-based WM agent that integrates: (1) a flexible tokenization framework supporting arbitrary combinations of observation and action modalities; (2) intrinsic motivation for epistemic uncertainty reduction; (3) prioritized world model replay; and (4) regression-as-classification for reward and return prediction. Simulus achieves state-of-the-art sample efficiency for planning-free WMs across three diverse benchmarks: visual Atari 100K, continuous-control DMC Proprioception 500K, and symbolic Craftax-1M. Notably, intrinsic motivation proves beneficial even under the tight interaction budgets of sample-efficient RL, despite the risk of wasting scarce interactions on task-irrelevant experience. Ablation studies reveal that each component contributes individually, and their combination yields synergistic gains. Our code and model weights are publicly available at `https://github.com/leor-c/Simulus`.

## 1 Introduction

Sample efficiency refers to the ability of a reinforcement learning (RL) algorithm to learn effective policies using as few environment interactions as possible. In many real-world domains such as robotics, autonomous driving, and healthcare, this is particularly critical, as interactions are costly, slow, or constrained. World model agents, methods that learn control entirely from simulated experience generated by a learned dynamics model, have emerged as a promising approach to improving sample efficiency [Ha and Schmidhuber, 2018, Hafner et al., 2023, Micheli et al., 2023, Wang et al., 2024a].

Among world model agents, token-based world models (TBWMs) [Micheli et al., 2023, 2024, Cohen et al., 2024] are a particularly promising direction because they cast environment dynamics as sequence modeling over discrete tokens. This formulation aligns naturally with the success of large-scale sequence models, where multi-token representations have proven effective for modeling complex data. Evidently, most large scale world models [Agarwal et al., 2025, Decart et al., 2024, DeepMind, 2024] operate on multi-token observations, suggesting that such representations are advantageous at scale. TBWMs offer a clear modular design, separating the optimization of its representation, dynamics, and control models. As modular systems, TBWMs are easier to scale, develop, study, and deploy, as individual modules can be treated independently without interfering and are easier to master through divide and conquer. In addition, such separation leads to simpler optimization objectives and avoids interference between them (Appendix B.1).

Preprint.

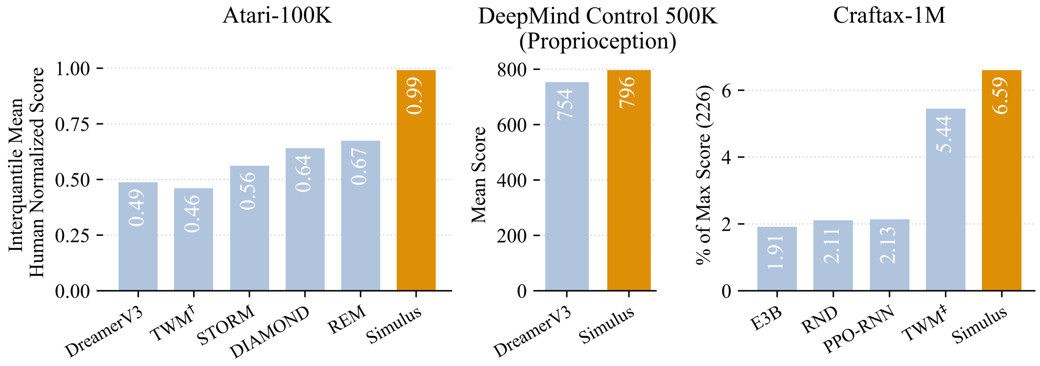


Figure 1: Results overview. Simulus exhibits state-of-the-art sample-efficiency performance for planning-free methods across all three benchmarks. † [Robine et al., 2023], ‡ [Dedieu et al., 2025].

Despite this promise, the potential of TBWMs remains largely unrealized. First, existing TBWMs are restricted to image observations and discrete actions, limiting both their adoption and broader applicability. While multi-modality tokenization approaches exist for large-scale offline settings [Reed et al., 2022, Schubert et al., 2023, Lu et al., 2023, 2024], these methods rely on large vocabularies (e.g., 33K tokens) which are inefficient for online, data-limited regimes. Whether substantially smaller vocabularies can preserve competitive performance remains an open question. Second, despite compelling results [Schmidhuber, 2010, Sekar et al., 2020, Kauvar et al., 2023], intrinsic motivation and prioritized *world model* replay are often overlooked in sample-efficient WM agents [Hafner et al., 2021, 2023, Zhang et al., 2024, Cohen et al., 2024, Alonso et al., 2024], particularly in combination. We believe this is a consequence of the high cost of integrating and validating new components in tightly coupled systems. We conjecture that intrinsic motivation is further underused in sample-efficient settings, as it may steer the agent toward task-irrelevant regions and waste the limited interaction budget. Prioritized replay [Kauvar et al., 2023], while promising, lacks robust empirical support and proved challenging to tune.

This situation is reminiscent of the landscape that preceded Rainbow [Hessel et al., 2018]: a set of individually validated improvements that had nonetheless never been systematically combined. Rainbow demonstrated the benefits of combining such complementary improvements, and became a landmark in the deep RL literature. We ask whether the same principle applies to world model agents, and introduce Simulus, a modular token-based WM agent that integrates: (1) a tokenization framework for handling arbitrary combinations of observation and action modalities, (2) intrinsic motivation for epistemic uncertainty reduction [Shyam et al., 2019, Sekar et al., 2020], (3) prioritized world model replay [Kauvar et al., 2023], and (4) regression-as-classification (RaC) [Farebrother et al., 2024] with exponential binning [Hafner et al., 2023] for reward and return prediction.

To evaluate the impact of the proposed components, we conducted extensive empirical evaluations across three diverse benchmarks, ranging from the visual Atari 100K [Kaiser et al., 2020], to the continuous proprioception tasks of the DeepMind Control Suite [Tunyasuvunakool et al., 2020], to Craftax [Matthews et al., 2024], which combines symbolic 2D grid maps with continuous state features. Simulus achieves state-of-the-art sample efficiency for planning-free WMs across all three. Ablation studies confirm the individual contribution of each component, and reveal considerable synergies when they are combined.

Summary of contributions:

- We identify and integrate promising but overlooked components (intrinsic motivation, prioritized replay, and a hybrid RaC), showing across three diverse benchmarks that each contributes significantly, with particularly strong gains when combined.
- We establish TBWMs as versatile agents by adapting multi-modality tokenization to the sample-efficient regime, with Simulus achieving state-of-the-art planning-free performance across visual, continuous, and multi-modal benchmarks.

- We show that world-model-induced exploration significantly improves performance even under tight interaction budgets, despite the risk of wasting scarce interactions on task-irrelevant experience.
- A strong starting point for TBWM research: building a competitive WM agent with all the integrated components requires significant expertise, engineering effort, and careful tuning. We open-source our code and model weights to facilitate future research.

## 2 Method

**Notations** We consider the Partially Observable Markov Decision Process (POMDP) setting. However, since in practice the agent has no knowledge about the hidden state space, consider the following state-agnostic formulation. Let $\Omega, \mathcal{A}$ be the sets of observations and actions, respectively. At every step $t$, the agent observes $\mathbf{o}_t \in \Omega$ and picks an action $a_t \in \mathcal{A}$. From the agent's perspective, the environment evolves according to $\mathbf{o}_{t+1}, r_t, d_t \sim p(\mathbf{o}_{t+1}, r_t, d_t | \mathbf{o}_{\leq t}, \mathbf{a}_{\leq t})$, where $r_t, d_t$ are the observed reward and termination signals, respectively. The process repeats until a positive termination signal $d_t \in \{0, 1\}$ is obtained. The agent's objective is to maximize its expected return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_{t+1}]$ where $\gamma \in [0, 1]$ is a discount factor.

For multi-modal observations, let $\mathbf{o}_t = \{\mathbf{o}_t^{(i)}\}_{i=1}^{|\kappa|}$ where $\kappa$ is the set of environment modalities and $\mathbf{o}_t^{(i)}$ denotes the features of modality $\kappa_i$.

**Overview** Simulus builds on REM [Cohen et al., 2024]. The agent comprises a representation module $\mathcal{V}$, a world model $\mathcal{M}$, a controller $\mathcal{C}$, and a replay buffer. To facilitate a modular design, following REM, each module is optimized separately. The training process of the agent involves a repeated cycle of four steps: data collection, representation learning ($\mathcal{V}$), world model learning ($\mathcal{M}$), and control learning in imagination ($\mathcal{C}$).

### 2.1 The Representation Module $\mathcal{V}$

$\mathcal{V}$ is responsible for encoding and decoding raw observations and actions. It is a modular tokenization system with encoder-decoder pairs for different input modalities. Encoders produce fixed-length token sequences, creating a common interface that enables combining tokens from various sources into a unified representation. After embedding, these token sequences are concatenated into a single representation, as described in Section 2.2. Note that encoder-decoder pairs need not be learning-based methods; however, when learned, they are optimized independently. This design enables $\mathcal{V}$ to deal with any combination of input modalities, provided the respective encoder-decoder pairs.

**Tokenization** $\mathcal{V}$ transforms raw observations $\mathbf{o}$ to sets of fixed-length integer token sequences $\mathbf{z} = \{\mathbf{z}^{(i)}\}_{i=1}^{|\kappa|}$ by applying the encoder of each modality $\mathbf{z}^{(i)} = \text{enc}_i(\mathbf{o}^{(i)})$. Actions $a$ are tokenized using the encoder-decoder pair of the related modality to produce $\mathbf{z}^{\text{a}}$. The respective decoders reconstruct observations from their tokens: $\hat{\mathbf{o}}^{(i)} = \text{dec}_i(\mathbf{z}^{(i)})$.

Simulus natively supports four modalities: images, continuous vectors, categorical variables, and image-like multi-channel grids of categorical variables, referred to as "2D categoricals". More formally, 2D categoricals are elements of $([k_1] \times [k_2] \times \ldots \times [k_C])^{m \times n}$ where $k_1, \ldots, k_C$ are per channel vocabulary sizes, $C$ is the number of channels, $m, n$ are spatial dimensions, and $[k] = \{1, \ldots, k\}$. Further details on per-modality tokenization are available in Appendix A.3.2.

### 2.2 The World Model $\mathcal{M}$

The purpose of $\mathcal{M}$ is to learn a model of the environment's dynamics. Concretely, given trajectory segments $\tau_t = \mathbf{z}_1, \mathbf{z}_1^{\text{a}}, \ldots, \mathbf{z}_t, \mathbf{z}_t^{\text{a}}$ in token representation, $\mathcal{M}$ models the distributions of the next observation and termination signal, and the expected reward:

$$\text{Transition:} \quad p_\theta(\hat{\mathbf{z}}_{t+1} | \tau_t), \tag{1}$$

$$\text{Reward:} \quad \hat{r}_t = \hat{r}_\theta(\tau_t), \tag{2}$$

$$\text{Termination:} \quad p_\theta(\hat{d}_t | \tau_t), \tag{3}$$

where $\theta$ is the parameters vector of $\mathcal{M}$ and $\hat{r}_\theta(\tau_t)$ is an estimator of $\mathbb{E}_{r_t \sim p(r_t | \tau_t)}[r_t]$.

**Architecture** $\mathcal{M}$ comprises a sequence model $f_\theta$ and multiple heads for the prediction of tokens of different observation modalities, rewards, termination signals, and for the estimation of model uncertainty. Concretely, $f_\theta$ is a retentive network (RetNet) [Sun et al., 2023] augmented with a parallel observation prediction (POP) [Cohen et al., 2024] mechanism. All heads are implemented as multilayer perceptrons (MLP) with a single hidden layer. We defer the details about these architectures to Appendix A.4.

**Embedding** $\mathcal{M}$ translates token trajectories $\tau$ into sequences of $d$-dimensional embeddings $\mathbf{X}$ using a set of embedding (look-up) tables. By design, each modality is associated with a separate table. In cases where an embedding table is not provided by the appropriate encoder-decoder pair, $\mathcal{M}$ and $\mathcal{C}$ learn dedicated tables separately and independently. As embeddings sequences are composed hierarchically, we use the following hierarchical notation:

$$\text{Observation-action block:} \quad \mathbf{X}_t = (\mathbf{X}_t^{\mathrm{o}}, \mathbf{X}_t^{\mathrm{a}}),$$

$$\text{Observation block:} \quad \mathbf{X}_t^{\mathrm{o}} = (\mathbf{X}_t^{(1)}, \dots, \mathbf{X}_t^{(|\kappa|)}),$$

where $K_i$ denotes the number of embedding vectors in $\mathbf{X}_t^{(i)}$. Similarly, $K = \sum_{i=1}^{|\kappa|} K_i$. To combine latents of each $\mathbf{z}_t$, $\mathcal{V}$ concatenates their token sequences along the temporal axis based on a predefined modality order. We defer the full details on the embedding process to Appendix A.4.

**Sequence Modeling** Given a sequence of observation-action blocks $\mathbf{X} = \mathbf{X}_1, \dots, \mathbf{X}_t$, the matching outputs $\mathbf{Y}_1, \dots, \mathbf{Y}_t$ are computed auto-regressively as follows:

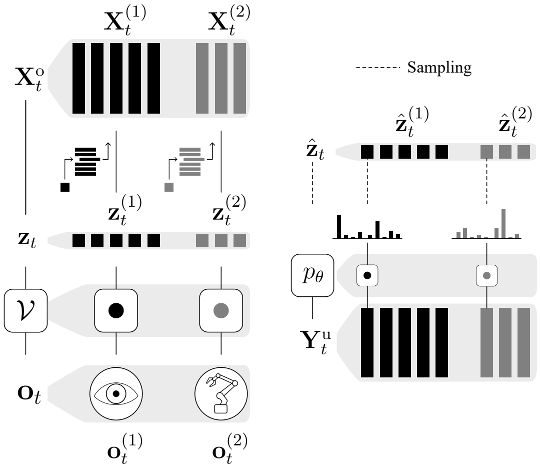
$$(\mathbf{S}_t, \mathbf{Y}_t) = f_\theta(\mathbf{S}_{t-1}, \mathbf{X}_t),$$

where $\mathbf{S}_t$ is a recurrent state that summarizes $\mathbf{X}_{\leq t}$ and $\mathbf{S}_0 = 0$. However, the output $\mathbf{Y}_{t+1}^{\mathrm{u}}$, from which $\hat{\mathbf{z}}_{t+1}$ is predicted, is computed using the POP mechanism via another call as

$$(\cdot, \mathbf{Y}_{t+1}^{\mathrm{u}}) = f_\theta(\mathbf{S}_t, \mathbf{X}^{\mathrm{u}}),$$

where $\mathbf{X}^{\mathrm{u}} \in \mathbb{R}^{K \times d}$ is a learned embedding sequence. Intuitively, $\mathbf{X}^{\mathrm{u}}$ acts as a learned prior, enabling the parallel generation of multiple tokens into the future.

To model $p_\theta(\hat{\mathbf{z}}_{t+1} | \mathbf{Y}_{t+1}^{\mathrm{u}})$, the distributions $p_\theta(\hat{z} | \mathbf{y})$ of each token $\hat{z}$ of $\hat{\mathbf{z}}_{t+1}^{(i)}$ of each modality $\kappa_i$ are modeled using modality-specific prediction heads implemented as MLPs with a single hidden layer and an output size equal to the vocabulary size of $\mathrm{enc}_i$ (Figure 2b). For 2D categoricals, $C$ heads are used to predict the $C$ tokens from each $\mathbf{y}$.



(a) Observation tokenization and embedding.

(b) Prediction of observation tokens.

Figure 2: An illustration of the independent processing of modalities for an observation with two modalities.

Similarly, rewards and termination signals are predicted by additional prediction heads as $\hat{r}_t = \hat{r}_\theta(\mathbf{y}), \hat{d}_t \sim p_\theta(\hat{d}_t | \mathbf{y})$, slightly abusing notations, where $\mathbf{y}$ is the last vector of $\mathbf{Y}_{t+1}^{\mathrm{u}}$. An illustration is provided in Figure 3.

**Reducing Epistemic Uncertainty via Intrinsic Motivation** The world model $\mathcal{M}$ serves as the cornerstone of the entire system. Any controller operating within a world model framework can only perform as well as the underlying world model allows, making its quality a fundamental limiting factor. In turn, the model's performance depends heavily on the quality of its training data. Effective modeling requires data that sufficiently covers the environment's state-action space, yet achieving this coverage presents a particular challenge in online RL, where the controller must systematically and efficiently explore its environment. Success depends on intelligently guiding the controller toward unexplored or undersampled regions of the dynamics space. An effective approach to this challenge involves estimating the world model's epistemic uncertainty and directing the controller to gather

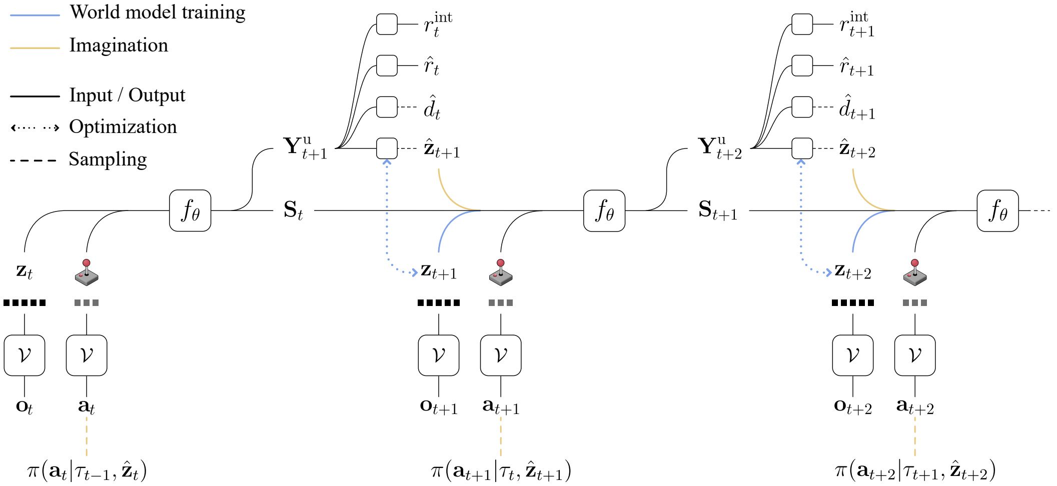

World model training
Imagination
Input / Output
Optimization
Sampling
$r_t^{\text{int}}$
$\hat{r}_t$
$\hat{d}_t$
$\hat{\mathbf{z}}_{t+1}$
$\mathbf{Y}^{\mathrm{u}}_{t+1}$
$f_\theta$
$\mathbf{S}_t$
$\mathbf{z}_t$
$\mathbf{z}_{t+1}$
$\mathcal{V}$
$\mathbf{o}_t$
$\mathbf{a}_t$
$\mathbf{o}_{t+1}$
$\mathbf{a}_{t+1}$
$\pi(\mathbf{a}_t|\tau_{t-1}, \hat{\mathbf{z}}_t)$
$\pi(\mathbf{a}_{t+1}|\tau_t, \hat{\mathbf{z}}_{t+1})$
$r_{t+1}^{\text{int}}$
$\hat{r}_{t+1}$
$\hat{d}_{t+1}$
$\hat{\mathbf{z}}_{t+2}$
$\mathbf{Y}^{\mathrm{u}}_{t+2}$
$\mathbf{S}_{t+1}$
$\mathbf{z}_{t+2}$
$\mathbf{o}_{t+2}$
$\mathbf{a}_{t+2}$
$\pi(\mathbf{a}_{t+2}|\tau_{t+1}, \hat{\mathbf{z}}_{t+2})$

Figure 3: World model training and imagination. To maintain visual clarity, we omitted token embedding details, as well as optimization details of rewards and termination signals.

data from regions where this uncertainty is highest [Schmidhuber, 2010, Shyam et al., 2019, Sekar et al., 2020].

Our approach estimates epistemic uncertainty using an ensemble of $N_{\text{ens}} = 4$ next observation prediction heads $\{p_{\phi_i}(\hat{\mathbf{z}}| \operatorname{sg}(\mathbf{Y}^{\mathrm{u}}))\}_{i=1}^{N_{\text{ens}}}$ with parameters $\{\phi_i\}_{i=1}^{N_{\text{ens}}}$ [Sekar et al., 2020, Lakshminarayanan et al., 2017] where $\operatorname{sg}(\cdot)$ is the stop gradient operator. To quantify disagreement between the ensemble's distributions, we employ the Jensen-Shannon divergence (JSD) [Shyam et al., 2019]. For probability distributions $P_1, \ldots, P_n$, the JSD is defined as:

$$\operatorname{JSD}(P_1, \ldots, P_n) = \mathcal{H}(\frac{1}{n}\sum_{i=1}^{n} P_i) - \frac{1}{n}\sum_{i=1}^{n} \mathcal{H}(P_i),$$

where $\mathcal{H}(\cdot)$ denotes the Shannon entropy. Since observations comprise multiple tokens, we average the per-token JSD values to obtain a single uncertainty measure $\delta_t$. Training data is divided equally among ensemble members, with each predictor processing a distinct subset of each batch. Despite the ensemble approach, our implementation maintains computational efficiency, with negligible additional overhead in practice.

To guide $\mathcal{C}$ towards regions of high epistemic uncertainty, $\mathcal{M}$ provides $\mathcal{C}$ with additional intrinsic rewards $r_t^{\text{int}} = \delta_t$ during imagination. Here, the reward provided by $\mathcal{M}$ at each step $t$ is given by

$$\bar{r}_t = w^{\text{int}} r_t^{\text{int}} + w^{\text{ext}} \hat{r}_t,$$

where $w^{\text{int}}, w^{\text{ext}} \in \mathbb{R}$ are hyperparameters that control the scale of each reward type. Optimizing the controller in imagination allows it to reach areas of high model uncertainty without additional real-environment interaction.

**Prioritized Replay** Recent work has demonstrated that prioritizing replay buffer sampling during world model training could lead to significant performance gains in intrinsically motivated agents [Kauvar et al., 2023]. While their approach showed promise, it required extensive hyperparameter tuning in practice. We propose a simpler, more robust prioritization scheme for world model training.

Here, the replay buffer maintains a world model loss value for each stored example, with newly added examples assigned a high initial loss value $\nu_0$. During $\mathcal{M}$'s training, we sample each batch using a mixture of uniform and prioritized sampling, controlled by a single parameter $\alpha \in [0, 1]$ that determines the fraction of prioritized samples. For the prioritized portion, we sample examples proportional to their softmax-transformed losses $p_i = \operatorname{softmax}(\mathcal{L})_i$. The loss values are updated after each world model optimization step using the examples' current batch losses.

**Training** We use the cross-entropy loss for the optimization of all components of $\mathcal{M}$. Specifically, for each $t$, the loss of $p_\theta(\hat{\mathbf{z}}_t|\mathbf{Y}^{\mathrm{u}}_t)$ is obtained by averaging the cross-entropy losses of its individual tokens. The same loss is used for each ensemble member $p_{\phi_i}(\hat{\mathbf{z}}_t | \operatorname{sg}(\mathbf{Y}_t^{\mathrm{u}}))$. The optimization and design of the reward predictor is similar to that of the critic, as described in Section 2.3. A formal description of the optimization objective can be found in Appendix A.4.1.

### 2.3 The Controller $\mathcal{C}$

$\mathcal{C}$ is an extended version of the actor-critic of REM [Cohen et al., 2024] that supports additional observation and action spaces and implements regression-as-classification for return predictions.

**Regression as Classification for Reward and Return Prediction** Robustly handling unbounded reward signals has long been challenging as they can vary dramatically in both magnitude and frequency. Hafner et al. [2023] addressed this challenge by using a classification network that predicts the weights of exponentially spaced bins and employed a two-hot loss for the network's optimization. Farebrother et al. [2024] studied the use of cross-entropy loss in place of the traditional mean squared error loss for value-based deep RL methods. In their work, the HL-Gauss method was shown to significantly outperform the two-hot loss method. Building on these developments, we adopt the classification network with exponential bins from [Hafner et al., 2023], and apply the HL-Gauss method for its optimization. Concretely, the critic's value estimates are predicted using a linear output layer parameterized by $\mathbf{W} \in \mathbb{R}^{m \times d_C}$ with $m = 128$ outputs corresponding to $m$ uniform bins defined by $m + 1$ endpoints $\mathbf{b} = (b_0, \dots, b_m)$. The predicted value is given by

$$\hat{y} = \operatorname{symexp}\left(\operatorname{softmax}(\mathbf{W}\mathbf{h})^\top \hat{\mathbf{b}}\right)$$

where $\mathbf{h}$ denotes the controller output latent representation (see Appendix A.5), $\operatorname{symexp}(x) = \operatorname{sign}(x)(\exp(|x|) - 1)$ is the inverse of the symlog function, and $\hat{\mathbf{b}} = \left(\frac{b_1+b_0}{2}, \dots, \frac{b_m+b_{m-1}}{2}\right)$ are the bin centers. Given the true target $y \in \mathbb{R}$, the HL-Gauss loss is given by

$$\mathcal{L}_{\text{HL-Gauss}}(\mathbf{W}, \mathbf{h}, y) = \tilde{\mathbf{y}}^\top \log \operatorname{softmax}(\mathbf{W}\mathbf{h})$$

where $\tilde{y}_i = \Phi(\frac{b_i - \operatorname{symlog}(y)}{\sigma}) - \Phi(\frac{b_{i-1} - \operatorname{symlog}(y)}{\sigma})$, $\Phi$ is the cumulative density function of the standard normal distribution and $\sigma$ is a standard deviation hyperparameter that controls the amount of label smoothing.

**Training in Imagination** $\mathcal{C}$ is trained entirely from simulated experience generated through interaction with $\mathcal{M}$. Specifically, $\mathcal{M}$ and $\mathcal{C}$ are initialized with a short trajectory segment sampled uniformly from the replay buffer and interact for $H$ steps. An illustration of this process is given in Figure 3 (orange path). $\lambda$-returns are computed for each generated trajectory segment and are used as targets for critic learning. For policy learning, a REINFORCE [Sutton et al., 1999] objective is used, with a $\hat{V}^\pi$ baseline for variance reduction. See Appendix A.5.2 for further details.

## 3 Experiments

To evaluate sample efficiency, we used benchmarks that measure performance within a fixed, limited environment interaction budget. These benchmarks were also selected to address key research questions: (1) whether the integrated components are individually effective and complementary across diverse environments; and (2) whether the proposed tokenization approach is effective in handling flexible combinations of observation and action modalities in sample-efficiency settings.

### 3.1 Experimental Setup

**Benchmarks** We evaluate Simulus on three sample-efficiency benchmarks with diverse observation and action modalities: Atari 100K [Kaiser et al., 2020], DMC Proprioception 500K [Tunyasuvunakool et al., 2020, Hafner et al., 2023], and Craftax-1M [Matthews et al., 2024], a partially observable 2D survival environment inspired by Minecraft. These correspond to visual discrete control, continuous control from proprioceptive inputs, and mixed 2D symbolic and continuous observations, respectively. Each benchmark imposes a strict interaction budget (100K, 500K, and 1M steps). Additional environment details are provided in Appendix A.

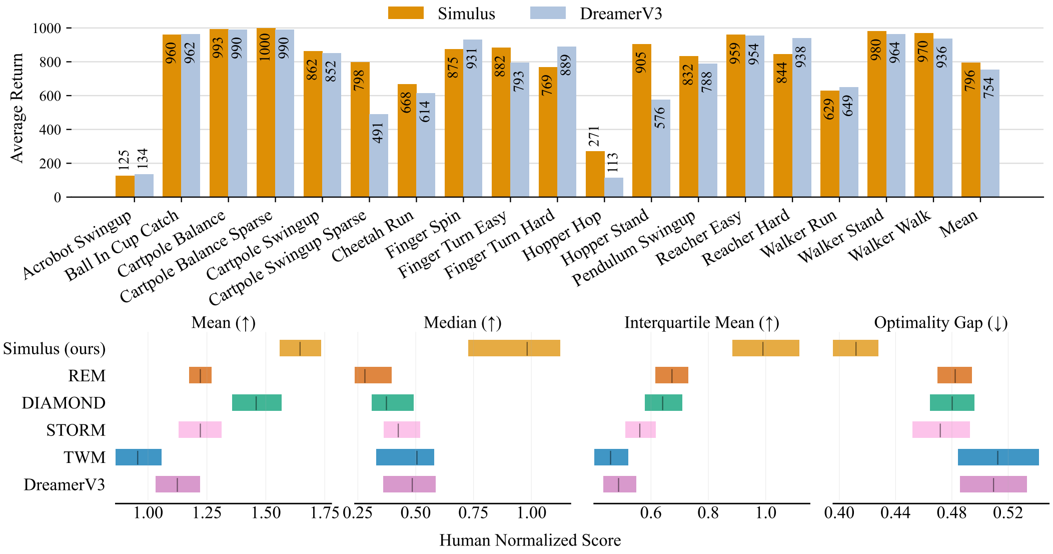


Figure 4: Results on the DeepMind Control Suite 500K Proprioception (top) and Atari 100K (bottom) benchmarks.

**Baselines** On Atari-100K, we compare Simulus against DreamerV3 [Hafner et al., 2023] and several methods restricted to image observations: TWM [Robine et al., 2023], STORM [Zhang et al., 2024], DIAMOND [Alonso et al., 2024], and REM [Cohen et al., 2024]. On DMC, we compare exclusively with DreamerV3, currently the only planning-free world model method with published results on the 500K proprioception benchmark. On Craftax-1M, we compare against TWM [Dedieu et al., 2025], a concurrent work that proposes a Transformer based world model with a focus on the Craftax benchmark, and the baselines reported in the Craftax paper: Random Network Distillation (RND) [Burda et al., 2019], PPO [Schulman et al., 2017] with a recurrent neural network (PPO-RNN), and Exploration via Elliptical Episodic Bonuses (E3B) [Henaff et al., 2022]. As Craftax is a recent benchmark, there are no other published results in existing world models literature. Following the standard practice in the literature, we exclude planning-based methods [Hansen et al., 2024, Wang et al., 2024a], as planning is an orthogonal component that operates on any given model, typically incurring significant computational overhead.

**Metrics and Evaluation** For Atari, we report human-normalized scores (HNS), calculated as $\frac{\text{agent\_score}-\text{random\_score}}{\text{human\_score}-\text{random\_score}}$ [Mnih et al., 2015]. Following the protocol of Agarwal et al. [2021] and using their toolkit, we report the mean, median, interquartile mean (IQM), and optimality gap metrics with 95% stratified bootstrap confidence intervals. For DMC and Craftax, we report the raw agent returns. We use 5 random seeds per environment. In each experiment, final performance is evaluated using 100 test episodes at the end of training and the mean score is reported.

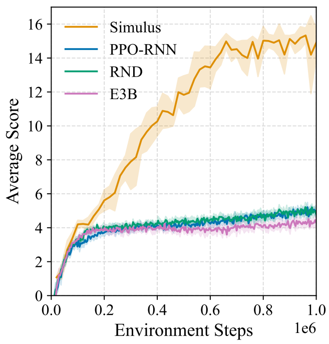


Figure 5: Craftax-1M training curves with mean and 95% confidence intervals.

## 3.2 Results

Simulus achieves state-of-the-art performance across all three benchmarks (Figure 1). On Atari 100k, it outperforms all baselines across key metrics (Figure 4). Notably, Simulus is the first planning-free world model to reach human-level IQM and median scores, achieving superhuman performance on **13** out of 26 games (Table 9, Appendix B). In Section 3.3, we show that these gains are attributed to the integration of the proposed components, demonstrating their individual and combined effectiveness.

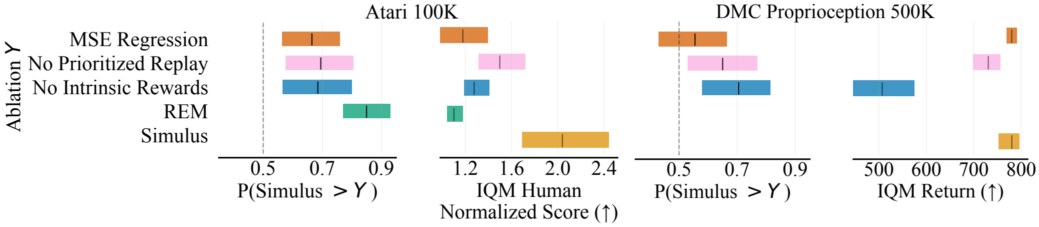


Figure 6: Ablations results on the Atari-100K and DeepMind Control Proprioception 500K benchmarks. A subset of 8 games was used for each ablation.

**Effectiveness in continuous control environments** Figure 4 provides compelling evidence that token-based architectures can perform well in continuous domains, even with compact vocabularies: Simulus consistently matches DreamerV3 across most tasks and slightly outperforms it on average.

**Effectiveness in environments with multi-modal observations** We evaluate multi-modality performance in Craftax, as it combines a symbolic image-like grid map with a vector of continuous features, involving multiple tokenizers ($\mathcal{V}$). Simulus maintains sample-efficiency in this multi-modal environment, outperforming both concurrent world model methods (Figure 1) and all model-free baselines (Figure 5), including exploration-focused algorithms. With 444 tokens per observation arranged into sequences of 147 embeddings, even short trajectories in Craftax contain thousands of tokens, demonstrating Simulus's efficient handling of long sequences. These findings indicate that the world model ($\mathcal{M}$) and controller ($\mathcal{C}$) maintain strong performance under multi-modal inputs when processed by the proposed modular tokenizer.

### 3.3 Ablation Studies

We ablate the intrinsic rewards, prioritized replay, and regression-as-classification to demonstrate their individual contributions to Simulus's performance. In each experiment, Simulus is modified by disabling a single component. Due to limited computational resources, we consider a subset of 8 tasks for each of the Atari and DMC benchmarks, and exclude Craftax from this analysis. For Atari 100K, we report results on games exhibiting significant performance variation. For DMC, we chose a subset that includes different embodiments. We defer the specific environment names to Appendix C.

The results are presented in Figure 6. Although all components contributed to Simulus's final performance, intrinsic rewards were especially crucial for achieving competitive performance in both benchmarks. Interestingly, the Atari 100K results indicate that combining all three components yields a significantly stronger algorithm. These findings also suggest that both prioritized world model replay and regression-as-classification enhance the effectiveness of intrinsic rewards.

More broadly, the results in Figure 6 demonstrate that encouraging the controller to explore regions of high epistemic uncertainty through intrinsic rewards significantly improves its performance in world model agents, even in reward-rich environments. This observation is non-trivial in a sample-efficient setting, where the limited interaction budget makes model-driven exploration particularly costly, as it consumes resources that could otherwise be used for task-related exploration during data collection. The latter type of exploration aims to collect new information about the true reward signal, which defines the task and its success metric. On the other hand, model-driven exploration may guide the controller towards environment regions that are irrelevant to the task at hand.

## 4 Related Work

**Offline Multi-Modal Methods** Large-scale token sequence models for multi-modal agent trajectories have been proposed in [Lu et al., 2023, 2024, Reed et al., 2022, Schubert et al., 2023]. Gato [Reed et al., 2022] and TDM [Schubert et al., 2023] tokenize inputs via predefined transformations, while Unified IO [Lu et al., 2023, 2024] leverages pretrained models. These methods do not learn control through RL but rely on expert data. They also use massive models with billions of parameters, large vocabularies, and significantly more data and compute than sample-efficient world models. Consequently, it remains unclear whether their design choices would be effective in online, sample-efficient settings with non-stationary and limited data.

**World Model Agents** Model-based agents that learn policies solely from simulated data generated by a learned world model were introduced by Ha and Schmidhuber [2018], followed by the influential Dreamer family [Hafner et al., 2020, 2021, 2023]. Dreamer jointly optimizes its representation and recurrent world models via a KL divergence between learned prior and posterior distributions, leading to interfering objectives (Appendix B.1) and a complex, monolithic architecture that complicates development and scaling [Wu et al., 2024]. With the rise of Transformer architectures in language modeling [Vaswani et al., 2017, Brown et al., 2020], Transformer-based Dreamer variants emerged [Zhang et al., 2024, Robine et al., 2023], alongside token-based world models (TBWMs) that treat trajectories as discrete token sequences [Micheli et al., 2023, Cohen et al., 2024]. However, these methods are limited to visual environments with discrete actions (e.g., Atari 100K), leaving their performance in other modalities uncertain. Recently, DIAMOND [Alonso et al., 2024], a diffusion world model inspired by advances in generative modeling [Rombach et al., 2022], was introduced. While it generates visually compelling outputs, it remains limited to visual domains.

**Intrinsically Motivated World Model Agents** Although intrinsic motivation (IM) has been extensively studied [Pathak et al., 2017, Burda et al., 2019, Hazan et al., 2019, Badia et al., 2020b,a], its use in world models typically involves an exploration pretraining phase followed by limited task-specific fine-tuning [Sekar et al., 2020, Mendonca et al., 2021, Kauvar et al., 2023]. While combining IM with prioritized replay has shown promise [Kauvar et al., 2023], it remains unexplored in standard sample-efficiency settings with external rewards.

**Large-Scale Video World Models** Building on recent advances in video generative modeling [Ho et al., 2022, Blattmann et al., 2023, Brooks et al., 2024], recent works have introduced large-scale video world models [DeepMind, 2024, Agarwal et al., 2025, Decart et al., 2024, Valevski et al., 2024], trained offline on extensive pre-collected data to predict future frames. However, these models do not address control learning, particularly RL. While recent efforts aim to bridge this gap [Wu et al., 2024], they remain confined to visual environments and lack comprehensive empirical evaluation.

## 5 Limitations and Future Work

We briefly highlight several limitations of this work. First, although the feature quantization approach for tokenizing continuous vectors showed promise, it leads to excessive sequence lengths. We believe that more efficient solutions can be found for dealing with continuous inputs. Second, due to the scarcity of rich multi-modal RL benchmarks, we could not extensively explore diverse modality combinations in our experiments. Lastly, token-based world model agents remain slower to train than other baselines in sample-efficient RL. Nonetheless, their modular design enables faster policy inference as the controller is independent of the world model.

## 6 Conclusions

Many promising improvements to world model agents remain unrealized, as combining them requires significant expertise and engineering effort. It is further unclear whether such improvements are even complementary, let alone synergistic. Inspired by Rainbow, we introduce Simulus: a modular token-based WM agent that combines multi-modality tokenization with promising but overlooked components: intrinsic motivation for epistemic uncertainty reduction, prioritized world model replay, and RaC with exponential binning for reward and return prediction.

Across visual (Atari 100K), continuous (DMC Proprioception 500K), and symbolic multi-modal (Craftax-1M) benchmarks, Simulus achieves state-of-the-art sample efficiency for planning-free WMs, establishing TBWMs as versatile and widely applicable methods. Ablation studies confirm that each component contributes individually and that their combination yields synergistic gains. Notably, world-model-induced exploration proves effective even under tight interaction budgets.

# References

Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320. Curran Associates, Inc., 2021. URL `https://proceedings.neurips.cc/paper_files/paper/2021/file/f514cec81cb148559cf475e7426eed5e-Paper.pdf`.

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *arXiv preprint arXiv:2405.12399*, 2024.

Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Zhaohan Daniel Guo, and Charles Blundell. Agent57: Outperforming the Atari human benchmark. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 507–517. PMLR, 13–18 Jul 2020a. URL `https://proceedings.mlr.press/v119/badia20a.html`.

Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Bilal Piot, Steven Kapturowski, Olivier Tieleman, Martin Arjovsky, Alexander Pritzel, Andrew Bolt, and Charles Blundell. Never give up: Learning directed exploration strategies. In *International Conference on Learning Representations*, 2020b. URL `https://openreview.net/forum?id=Sye57xStvB`.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.

Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024. URL `https://openai.com/research/video-generation-models-as-world-simulators`.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL `https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf`.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=H1lJJnR5Ym`.

Lior Cohen, Kaixin Wang, Bingyi Kang, and Shie Mannor. Improving token-based world models with parallel observation prediction. In *Forty-first International Conference on Machine Learning*, 2024. URL `https://openreview.net/forum?id=Lfp5Dk1xb6`.

Decart, Julian Quevedo, Quinn McIntyre, Spruce Campbell, Xinlei Chen, and Robert Wachen. Oasis: A universe in a transformer, 2024. URL `https://oasis-model.github.io/`.

Antoine Dedieu, Joseph Ortiz, Xinghua Lou, Carter Wendelken, Wolfgang Lehrach, J Swaroop Guntupalli, Miguel Lazaro-Gredilla, and Kevin Patrick Murphy. Improving transformer world models for data-efficient rl, 2025. URL `https://arxiv.org/abs/2502.01591`.

Google DeepMind. Genie 2: A large-scale foundation world model, 2024. URL `https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/`.

Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021.

Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taiga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal. Stop regressing: Training value functions via classification for scalable deep RL. In *Forty-first International Conference on Machine Learning*, 2024. URL `https://openreview.net/forum?id=dVpFKfqF3R`.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems 31*, pages 2451–2463. Curran Associates, Inc., 2018. URL `https://papers.nips.cc/paper/7512-recurrent-world-models-facilitate-policy-evolution`. `https://worldmodels.github.io`.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=S1lOTC4tDS`.

Danijar Hafner, Timothy P. Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL `https://openreview.net/forum?id=0oabwyZbOu`.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=Oxh5CstDJU`.

Elad Hazan, Sham Kakade, Karan Singh, and Abby Van Soest. Provably efficient maximum entropy exploration. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2681–2691. PMLR, 09–15 Jun 2019. URL `https://proceedings.mlr.press/v97/hazan19a.html`.

Mikael Henaff, Roberta Raileanu, Minqi Jiang, and Tim Rocktäschel. Exploration via elliptical episodic bonuses. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL `https://openreview.net/forum?id=Xg-yZos9qJQ`.

Dan Hendrycks and Kevin Gimpel. Bridging nonlinearities and stochastic regularizers with gaussian error linear units, 2017. URL `https://openreview.net/forum?id=Bk0MRI5lg`.

Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *32nd AAAI Conference on Artificial Intelligence, AAAI 2018*, 2018. doi: 10.1609/aaai.v32i1.11796.

Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II 14*, pages 694–711. Springer, 2016.

Łukasz Kaiser, Mohammad Babaeizadeh, Piotr Miłos, Błażej Osiński, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=S1xCPJHtDB`.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 13–18 Jul 2020. URL `https://proceedings.mlr.press/v119/katharopoulos20a.html`.

Isaac Kauvar, Chris Doyle, Linqi Zhou, and Nick Haber. Curious replay for model-based adaptation. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL `https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf`.

Anders Boesen Lindbo Larsen, Søren Kaae Sønderby, Hugo Larochelle, and Ole Winther. Autoencoding beyond pixels using a learned similarity metric. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1558–1566, New York, New York, USA, 20–22 Jun 2016. PMLR. URL `https://proceedings.mlr.press/v48/larsen16.html`.

Jiasen Lu, Christopher Clark, Rowan Zellers, Roozbeh Mottaghi, and Aniruddha Kembhavi. UNIFIED-IO: A unified model for vision, language, and multi-modal tasks. In *The Eleventh International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=E01k9048soZ`.

Jiasen Lu, Christopher Clark, Sangho Lee, Zichen Zhang, Savya Khosla, Ryan Marten, Derek Hoiem, and Aniruddha Kembhavi. Unified-io 2: Scaling autoregressive multimodal models with vision language audio and action. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26439–26455, June 2024.

Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Jackson, Samuel Coward, and Jakob Foerster. Craftax: A lightning-fast benchmark for open-ended reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.

Russell Mendonca, Oleh Rybkin, Kostas Daniilidis, Danijar Hafner, and Deepak Pathak. Discovering and achieving goals via world models. *Advances in Neural Information Processing Systems*, 34: 24379–24391, 2021.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL `https://openreview.net/pdf?id=vhFu1Acb0xb`.

Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL `https://openreview.net/forum?id=BiWIERWBFX`.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787. PMLR, 06–11 Aug 2017. URL `https://proceedings.mlr.press/v70/pathak17a.html`.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions, 2018. URL `https://openreview.net/forum?id=SkBYYyZRZ`.

Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barth-maron, Mai Giménez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. A generalist agent. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL `https://openreview.net/forum?id=1ikK0kHjvj`. Featured Certification, Outstanding Certification.

Jan Robine, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. Transformer-based world models are happy with 100k interactions. *arXiv preprint arXiv:2303.07109*, 2023.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022.

Jürgen Schmidhuber. Formal theory of creativity, fun, and intrinsic motivation (1990-2010). *IEEE Transactions on Autonomous Mental Development*, 2, 2010. ISSN 19430604. doi: 10.1109/TAMD.2010.2056368.

Ingmar Schubert, Jingwei Zhang, Jake Bruce, Sarah Bechtle, Emilio Parisotto, Martin Riedmiller, Jost Tobias Springenberg, Arunkumar Byravan, Leonard Hasenclever, and Nicolas Heess. A generalist dynamics model for control. *arXiv preprint arXiv:2305.10912*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. Planning to explore via self-supervised world models. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8583–8592. PMLR, 13–18 Jul 2020. URL `https://proceedings.mlr.press/v119/sekar20a.html`.

Pranav Shyam, Wojciech Jaśkowski, and Faustino Gomez. Model-based active exploration. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5779–5788. PMLR, 09–15 Jun 2019. URL `https://proceedings.mlr.press/v97/shyam19a.html`.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL `https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf`.

Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020. ISSN 2665-9638. doi: https://doi.org/10.1016/j.simpa.2020.100022. URL `https://www.sciencedirect.com/science/article/pii/S2665963820300099`.

Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. *CoRR*, abs/2408.14837, 2024. URL `https://doi.org/10.48550/arXiv.2408.14837`.

Aaron van den Oord, Oriol Vinyals, and koray kavukcuoglu. Neural discrete representation learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL `https://proceedings.neurips.cc/paper_files/paper/2017/file/7a98af17e63a0ac09ce2e96d03992fbc-Paper.pdf`.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL `https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf`.

Shengjie Wang, Shaohuai Liu, Weirui Ye, Jiacheng You, and Yang Gao. Efficientzero v2: Mastering discrete and continuous control with limited data. In *Forty-first International Conference on Machine Learning*, 2024a. URL `https://openreview.net/forum?id=LHGMXcr6zx`.

ZiRui Wang, Yue DENG, Junfeng Long, and Yin Zhang. Parallelizing model-based reinforcement learning over the sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL `https://openreview.net/forum?id=R6N9AGyz13`.

Jialong Wu, Shaofeng Yin, Ningya Feng, Xu He, Dong Li, Jianye HAO, and Mingsheng Long. ivideoGPT: Interactive videoGPTs are scalable world models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL `https://openreview.net/forum?id=4TENzBftZR`.

Weipu Zhang, Gang Wang, Jian Sun, Yetian Yuan, and Gao Huang. Storm: Efficient stochastic transformer based world models for reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.

# A Experimental Setup

## A.1 Benchmarks

We evaluate Simulus on three sample-efficiency benchmarks of different observation and action modalities: Atari 100K [Kaiser et al., 2020], DeepMind Control Suite (DMC) Proprioception 500K [Tunyasuvunakool et al., 2020], and Craftax-1M [Matthews et al., 2024].

Atari 100K has become the gold standard in the literature for evaluating sample-efficient deep RL agents. The benchmark comprises a subset of 26 games. Within each game, agents must learn from visual image signal under a tightly restricted budget of 100K interactions, corresponding to roughly two hours of human gameplay.

The DeepMind Control Suite (DMC) comprises continuous control tasks ranging from simple single-joint embodiments to complex humanoids. We follow the subset of proprioception tasks used to evaluate DreamerV3 [Hafner et al., 2023], featuring continuous observation and action vectors. Each task limits the agent's interaction budget to 500K steps.

Craftax is a 2D open-world survival game benchmark inspired by Minecraft, designed to evaluate RL agents' capabilities in planning, memory, and exploration. The partially-observable environment features procedurally generated worlds where agents must gather and craft resources while surviving against hostile creatures. Observations consist of a 9×11 tile egocentric map, where each tile consists of 4 symbols, and 48 state features corresponding to state information such as inventory and health. Here, we consider the sample-efficiency oriented Craftax-1M variant which only allows an interaction budget of one million steps.

## A.2 Hyperparameters

We detail shared hyperparameters in Table 1, training hyperparameters in Table 2, world model hyperparameters in Table 3, and controller hyperparameters in Table 4. Environment hyperparameters are detailed in Table 5 (Atari-100K) and Table 6 (DMC).

For the DMC benchmark, we use a lower embedding dimension (Table 3) due to the significantly lower dimensionality of its observations compared to other benchmarks. As in prior work (e.g., DreamerV3), we adopt a smaller model for this setting. Note that the reduced number of Retention heads ensures a consistent head dimensionality (64).

Additionally, we used a limited decay range in DMC (Table 3) as observations effectively represent full MDP states, eliminating the need for long-term memory. By constraining the decay range, we explicitly encode this inductive bias into the model.

In the Craftax benchmark, we reduce the number of layers (Table 3) to lower computational cost. The interaction budget in Craftax is 1M steps, resulting in a particularly expensive training process. Here, we increase the decay range as an inductive bias to encourage long-term memory.

The weighting of intrinsic versus extrinsic rewards (Table 4) varies across benchmarks due to differences in reward structure and scale. For instance, DMC provides dense rewards with typical task scores reaching around 1000, whereas Craftax has extremely sparse rewards, with cumulative scores rarely exceeding 20—even over thousands of steps.

**Tuning** All remaining hyperparameters were tuned empirically or based on REM [Cohen et al., 2024], with minimal impact on training cost and adjusted primarily for performance. Due to limited computational resources, we were unable to conduct extensive tuning, and we believe that further optimization could improve Simulus's performance.

Table 1: Shared hyperparameters.

| Description | Symbol | Value |
|---|---|---|
| Eval sampling temperature | | 0.5 |
| Optimizer | | AdamW |
| Learning rate ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (1e-4, 2e-4, 2e-4) |
| AdamW $\beta_1$ | | 0.9 |
| AdamW $\beta_2$ | | 0.999 |
| Gradient clipping threshold ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (10, 3, 3) |
| Weight decay ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (0.01, 0.05, 0.01) |
| Prioritized replay fraction | $\alpha$ | 0.3 |
| Prioritized replay initial loss value | $\nu_0$ | 10 |
| $\mathcal{M}$ ensemble size | $N_{\text{ens}}$ | 4 |
| HL-Gauss num bins | | 128 |
| Label smoothing | $\sigma$ | $\frac{3}{4}$bin_width $= 0.1758$ |

Table 2: Training hyperparameters.

| Description | Symbol | Atari-100K | DMC | Craftax |
|---|---|---|---|---|
| Horizon | $H$ | 10 | 20 | 20 |
| Observation sequence length | $K$ | 64 | 3-24 | 147 |
| Action sequence length | $K_{\text{a}}$ | 1 | 1-6 | 1 |
| Tokenizer vocabulary size | $N$ | 512 | 125 | (37,5,40,20,4,125) |
| Epochs | | 600 | 1000 | 10000 |
| Experience collection epochs | | 500 | 1000 | 10000 |
| Environment steps per epoch | | 200 | 500 | 100 |
| Batch size ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (128, 32, 128) | (-, 16, 128) | (-, 8, 128) |
| Training steps per epoch ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (200, 200, 80) | (-, 300, 100) | (-, 100, 50) |
| Training start after epoch ($\mathcal{V}$, $\mathcal{M}$, $\mathcal{C}$) | | (5, 25, 50) | (-, 15, 20) | (-, 250, 300) |

Table 3: World model ($\mathcal{M}$) hyperparameters.

| Description | Symbol | Atari-100K | DMC | Craftax |
|---|---|---|---|---|
| Number of layers | | 10 | 10 | 5 |
| Number of heads | | 4 | 3 | 4 |
| Embedding dimension | $d$ | 256 | 192 | 256 |
| Dropout | | 0.1 | 0.1 | 0.1 |
| Retention decay range | $[\eta_{\min}, \eta_{\max}]$ | [4, 16] | [2, 2] | [8, 40] |

Table 4: Actor-critic ($\mathcal{C}$) hyperparameters.

| Description | Symbol | Atari-100K | DMC | Craftax |
|---|---|---|---|---|
| Environment reward weight | $w^{\text{ext}}$ | 1 | 1 | 100 |
| Intrinsic reward weight | $w^{\text{int}}$ | 1 | 10 | 1 |
| Encoder MLP ($g_\psi$) hidden layer sizes | | [512] | [384] | [512, 512] |
| Shared backbone | | True | False | True |
| Number of quantization values (continuous actions) | | | 51 | |
| (2D) Categoricals embedding dimension | | | | 64 |

Table 5: Atari 100K hyperparameters.

| Description | Symbol | Value |
|---|---|---|
| Frame resolution | | $64 \times 64$ |
| Frame Skip | | 4 |
| Max no-ops (train, test) | | (30, 1) |
| Max episode steps (train, test) | | (20K, 108K) |
| Terminate on live loss (train, test) | | (No, Yes) |

Table 6: DeepMind Control Suite Proprioception hyperparameters.

| Description | Symbol | Value |
|---|---|---|
| Action repeat | | 2 |

## A.3 The Representation Module $\mathcal{V}$

### A.3.1 Image Observations

Image observations are tokenized using a vector-quantized variational auto-encoder (VQ-VAE) [van den Oord et al., 2017, Esser et al., 2021]. A VQ-VAE comprises a convolutional neural network (CNN) encoder, an embedding table $\mathbf{E} \in \mathbb{R}^{n \times d}$, and a CNN decoder. Here, the size of the embedding table $n$ determines the vocabulary size.

The encoder's output $\mathbf{h} \in \mathbb{R}^{W \times H \times d}$ is a grid of $W \times H$ multi-channel vectors of dimension $d$ that encode high-level learned features. Each such vector is mapped to a discrete token by finding the closest embedding in $\mathbf{E}$:

$$z = \arg\min_i \|\mathbf{h} - \mathbf{E}(i)\|,$$

where $\mathbf{E}(i)$ is the $i$-th row of $\mathbf{E}$. To reconstruct the original image, the decoder first maps $\mathbf{z}$ to their embeddings using $\mathbf{E}$. During training, the straight-through estimator [Bengio et al., 2013] is used for backpropagating the learning signal from the decoder to the encoder: $\hat{\mathbf{h}} = \mathbf{h} + \mathrm{sg}(\mathbf{E}_z - \mathbf{h})$. The architecture of the encoder and decoder models is presented in Table 7.

The optimization objective is given by

$$\mathcal{L}(\mathrm{enc}, \mathrm{dec}, \mathbf{E}) = \\ \|\mathbf{o} - \mathrm{dec}(z)\|_2^2 + \|\,\mathrm{sg}(\mathrm{enc}(\mathbf{o})) - \mathbf{E}(z)\|_2^2 + \|\,\mathrm{sg}(\mathbf{E}(z) - \mathrm{enc}(\mathbf{o})\|_2^2 + \mathcal{L}_{\text{perceptual}}(\mathbf{o}, \mathrm{dec}(z)),$$

where $\mathcal{L}_{\text{perceptual}}$ is a perceptual loss [Johnson et al., 2016, Larsen et al., 2016], proposed in Micheli et al. [2023].

Crucially, the learned embedding table $\mathbf{E}$ is used for embedding the (image) tokens across all stages of the algorithm.

### A.3.2 Continuous Vectors

For the tokenization of continuous vectors, unbounded inputs are first transformed using the symlog function [Hafner et al., 2023], defined as:

$$\mathrm{symlog}(x) = \mathrm{sign}(x) \ln(1 + |x|).$$

This transformation compresses the magnitude of large absolute values. Then, each feature is quantized to produce a discrete token, following [Reed et al., 2022].

To improve learning efficiency, we reduce the vocabulary size from 1024 to 125 and modify the quantization levels for optimal coverage. Specifically, the quantization of each feature uses 125 values in the range $[-6, 6]$, where 63 values are uniformly distributed in $[-\ln(1+\pi), \ln(1+\pi)]$ and the rest are uniformly distributed in the remaining intervals.

Lastly, while no special tokenization is required for categorical inputs, 2D categoricals are flattened along the spatial dimensions to form a sequence of categorical vectors. The embedding of each token vector is obtained by averaging the embeddings of its entries.

Table 7: The encoder and decoder architectures of the VQ-VAE model. “Conv(a,b,c)" represents a convolutional layer with kernel size $a \times a$, stride of $b$ and padding $c$. A value of $c =$ Asym. represents an asymmetric padding where a padding of 1 is added only to the right and bottom ends of the image tensor. “GN" represents a GroupNorm operator with 8 groups, $\epsilon = 1e - 6$ and learnable per-channel affine parameters. SiLU is the Sigmoid Linear Unit activation [Hendrycks and Gimpel, 2017, Ramachandran et al., 2018]. “Interpolate" uses PyTorch’s interpolate method with scale factor of 2 and the “nearest-exact" mode.

| Module | Output Shape |
|---|---|
| Encoder | |
| Input | $3 \times 64 \times 64$ |
| Conv(3, 1, 1) | $64 \times 64 \times 64$ |
| EncoderBlock1 | $128 \times 32 \times 32$ |
| EncoderBlock2 | $256 \times 16 \times 16$ |
| EncoderBlock3 | $512 \times 8 \times 8$ |
| GN | $512 \times 8 \times 8$ |
| SiLU | $512 \times 8 \times 8$ |
| Conv(3, 1, 1) | $256 \times 8 \times 8$ |
| EncoderBlock | |
| Input | $c \times h \times w$ |
| GN | $c \times h \times w$ |
| SiLU | $c \times h \times w$ |
| Conv(3, 2, Asym.) | $2c \times \frac{h}{2} \times \frac{w}{2}$ |
| Decoder | |
| Input | $256 \times 8 \times 8$ |
| BatchNorm | $256 \times 8 \times 8$ |
| Conv(3, 1, 1) | $256 \times 8 \times 8$ |
| DecoderBlock1 | $128 \times 16 \times 16$ |
| DecoderBlock2 | $64 \times 32 \times 32$ |
| DecoderBlock3 | $64 \times 64 \times 64$ |
| GN | $64 \times 64 \times 64$ |
| SiLU | $64 \times 64 \times 64$ |
| Conv(3, 1, 1) | $3 \times 64 \times 64$ |
| DecoderBlock | |
| Input | $c \times h \times w$ |
| GN | $c \times h \times w$ |
| SiLU | $c \times h \times w$ |
| Interpolate | $c \times 2h \times 2w$ |
| Conv(3, 1, 1) | $\frac{c}{2} \times 2h \times 2w$ |

### A.4 The World Model $\mathcal{M}$

**Embedding Details** Each token in $\mathbf{z}^{(i)}$ of each modality is mapped to a $d$-dimensional embedding vector $\mathbf{X}^{(i)}$ using the embedding (look-up) table $\mathbf{E}^{(i)}$ of modality $\kappa_i$. The embedding vector that corresponds to token $z$ is simply the $z$-th row in the embedding table. Formally, $\mathbf{x}_{t,j}^{(i)} = \mathbf{E}^{(i)}(l),\ l = z_{t,j}^{(i)}$ where $\mathbf{E}(l)$ refers to the $l$-th row in $\mathbf{E}$. In the special case of 2D categorical inputs, $\mathbf{x}_{t,j}^{(i)} = \frac{1}{C}\sum_{n=1}^{C}\mathbf{E}_n^{(i)}(l_n),\ l_n = z_{t,j,n}^{(i)}$ where $C$ is the number of channels and $i$ is the index of the 2D categorical modality in $\kappa$.

To concatenate the embeddings, we use the following order among the modalities: images, continuous vectors, categorical variables, and 2D categoricals.

**Prediction Heads** Each prediction head in $\mathcal{M}$ is a multi-layer perceptron (MLP) with a single hidden layer of dimension $2d$ where $d$ is the embedding dimension.

**Epistemic Uncertainty Estimation** Working with discrete distributions enables efficient entropy computation and ensures that the ensemble disagreement term $\delta_t$ is bounded by $\frac{1}{|\mathbf{z}_t|}\sum_{z \in \mathbf{z}_t} \log(\text{vocab\_size}(z))$.

#### A.4.1 Optimization

For each training example in the form of a trajectory segment in token representation $\tau = \mathbf{z}_1, \mathbf{z}_1^{\text{a}}, \dots, \mathbf{z}_H, \mathbf{z}_H^{\text{a}}$, the optimization objective is given by

$$\mathcal{L}_{\mathcal{M}}(\theta, \phi, \tau) = \sum_{t=1}^{H} \mathcal{L}_{\text{obs}}(\theta, \mathbf{z}_t, p_\theta(\hat{\mathbf{z}}_t|\mathbf{Y}_t^{\text{u}})) + \mathcal{L}_{\text{reward}}(\theta, r_t, \hat{r}_t) - \log(p_\theta(d_t|\mathbf{Y}_t^{\text{u}}))$$
$$+ \sum_{i=1}^{N_{\text{ens}}} \mathcal{L}_{\text{obs}}(\phi_i, \mathbf{z}_t, p_{\phi_i}(\hat{\mathbf{z}}_t|\,\text{sg}(\mathbf{Y}_t^{\text{u}}))),$$

where

$$\mathcal{L}_{\text{obs}}(\theta, \mathbf{z}_t, p_\theta(\hat{\mathbf{z}}_t|\mathbf{Y}_t^{\text{u}})) = -\frac{1}{K}\sum_{i=1}^{K} \log p_\theta(z_i|\mathbf{y}_i)$$

is the average of the cross-entropy losses of the individual tokens, and $\mathcal{L}_{\text{reward}}(\theta, r_t, \hat{r}_t)$ is the $\mathcal{L}_{\text{HL-Gauss}}$ loss with the respective parameters of the reward head. Here, $\mathbf{y}_i$ is the vector of $\mathbf{Y}_t^{\text{u}}$ that corresponds to $z_i$, the $i$-th token of $\mathbf{z}_t$.

#### A.4.2 Retentive Networks

Retentive Networks (RetNet) [Sun et al., 2023] are sequence model architectures with a Transformer-like structure [Vaswani et al., 2017]. However, RetNet replaces the self-attention mechanism with a linear-attention [Katharopoulos et al., 2020] based Retention mechanism. At a high level, given an input sequence $\mathbf{X} \in \mathbb{R}^{|\mathbf{X}| \times d}$ of $d$-dimensional vectors, the Retention operator outputs

$$\text{Retention}(\mathbf{X}) = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{D})\mathbf{V},$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are the queries, keys, and values, respectively, and $\mathbf{D}$ is a causal mask and decay matrix. Notably, the softmax operation is discarded in Retention and other linear attention methods. As a linear attention method, the computation can also be carried in a recurrent form:

$$\text{Retention}(\mathbf{x}_t, \mathbf{S}_{t-1}) = \mathbf{S}_t \mathbf{q}_t,$$
$$\mathbf{S}_t = \eta \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d \times d},$$

where $\eta$ is a decay factor, $\mathbf{S}_t$ is a recurrent state, and $\mathbf{S}_0 = 0$. In addition, a hybrid form of recurrent and parallel forward computation known as the chunkwise mode allows to balance the quadratic cost of the parallel form and the sequential cost of the recurrent form by processing the input as a sequence of chunks. We refer the reader to Sun et al. [2023] for the full details about this architecture.

In our implementation, since inputs are complete observation-action block sequences $\mathbf{X}_1, \dots, \mathbf{X}_t$, we configure the decay factors of the multi-scale retention operator in block units:

$$\eta = 1 - 2^{-\operatorname{linspace}(\log_2(K\eta_{\text{min}}), \log_2(K\eta_{\text{max}}), N_h)},$$

where $\operatorname{linspace}(a, b, c)$ is a sequence of $c$ values evenly distributed between $a$ and $b$, $N_h$ is the number of retention heads, and $\eta_{\text{min}}, \eta_{\text{max}}$ are hyperparameters that control the memory decay in observation-action block units.

### A.4.3 Parallel Observation Prediction (POP)

POP [Cohen et al., 2024] is a mechanism for parallel generation of non-causal subsequences such as observations in token representation. It's purpose is to improve generation efficiency by alleviating the sequential bottleneck caused by generating observations a single token at a time (as done in language models). However, to achieve this goal, POP also includes a mechanism for maintaining training efficiency. Specifically, POP extends the chunkwise forward mode of RetNet to maintain efficient training of the sequence model.

To generate multiple tokens into the future at once, POP introduces a set of prediction tokens $\mathbf{u} = u_1, \dots, u_K$ and embeddings $\mathbf{X}^{\text{u}} \in \mathbb{R}^{K \times d}$ where $K$ is the number of tokens in an observation. Each token in $\mathbf{u}$ corresponds to an observation token in $\mathbf{z}$. These tokens, and their respective learned embeddings, serve as a learned prior.

Let $\mathbf{X}_1, \dots, \mathbf{X}_T$ be a sequence of $T$ observation-action (embeddings) blocks. Given $\mathbf{S}_{t-1}$ summarizing all key-value outer products of elements of $\mathbf{X}_{\leq t-1}$, the outputs $\mathbf{Y}^{\text{u}}$ from which the next observation tokens are predicted are given by:

$$(\cdot, \mathbf{Y}^{\text{u}}_t) = f_\theta(\mathbf{S}_{t-1}, \mathbf{X}^{\text{u}}).$$

Importantly, the recurrent state is never updated based on the prediction tokens $\mathbf{u}$ (or their embeddings). The next observation tokens $\hat{\mathbf{z}}_t$ are sampled from $p_\theta(\hat{\mathbf{z}}_t | \mathbf{Y}^{\text{u}}_t)$. Then, the next action is generated by the controller, and the next observation-action block $\mathbf{X}_t$ can be processed to predict the next observation $\hat{\mathbf{z}}_{t+1}$.

To maintain efficient training, a two step computation is carried at each RetNet layer. First, all recurrent states $\mathbf{S}_t$ for all $1 \leq t \leq T$ are calculated in parallel. Although there is an auto-regressive relationship between time steps, the linear structure of $\mathbf{S}$ allows to calculate the compute-intensive part of each state in parallel and incorporate past information efficiently afterwards. In the second step, all outputs $\mathbf{Y}^{\text{u}}_t$ for all $1 \leq t \leq T$ are computed in parallel, using the appropriate states $\mathbf{S}_{t-1}$ and $\mathbf{X}^{\text{u}}$ in batch computation. Note that this computation involves delicate positional information handling. We refer the reader to Cohen et al. [2024] for full details of this computation.

### A.5 The Controller $\mathcal{C}$

**Architecture** At the core of $\mathcal{C}$'s architecture, parameterized by $\psi$, is an LSTM [Hochreiter and Schmidhuber, 1997] sequence model. At each step $t$, upon observing $\mathbf{z}_t$, a set of modality-specific encoders map each modality tokens $\mathbf{z}_t^{(i)}$ to a latent vector $\mathbf{x}^{(i)}$, where we abuse our notation $\mathbf{x}$ as the context of the discussion is limited to $\mathcal{C}$. The latents are then fused by a fully-connected network to obtain a single vector $\mathbf{x} = g_\psi(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(|\kappa|)})$. $\mathbf{x}_t \in \mathbb{R}^{dc}$ is processed by $\mathcal{C}$'s sequence model to produce $\mathbf{h}_t, \mathbf{c}_t = \mathrm{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \psi)$ where $\mathbf{h}_t, \mathbf{c}_t$ are the LSTM's hidden and cell states, respectively. Lastly, two linear output layers produce the logits from which the actor and critic outputs $\pi(a_t|\mathbf{h}_t), \hat{V}^\pi(\mathbf{h}_t)$ are derived. For continuous action spaces, the actor uses a categorical distribution over a uniformly spaced discrete subset of $[-1, 1]$.

**Critic** The value prediction uses 128 bins in the range $\mathbf{b} = (-15, \dots, 15)$.

**Continuous Action Spaces** The policy network outputs $m = 51$ logits corresponding to $m$ quantization values uniformly distributed in $[-1, 1]$ for each individual action in the action vector.

#### A.5.1 Input Encoding

The controller $\mathcal{C}$ operates in the latent token space. Token trajectories $\tau = \mathbf{z}_1, \mathbf{z}_1^{\mathrm{a}}, \dots, \mathbf{z}_H, \mathbf{z}_H^{\mathrm{a}}$ are processed sequentially by the LSTM model. At each time step $t$, the network gets $\mathbf{z}_t$ as input, outputs $\pi(a_t|\tau_{\leq t-1}, \mathbf{z}_t)$ and $\hat{V}^\pi(a_t|\tau_{\leq t-1}, \mathbf{z}_t)$, samples an action $a_t$ and then process the sampled action as another sequence element.

The processing of actions involve embedding the action into a latent vector which is then provided as input to the LSTM. Embedding of continuous action tokens is performed by first reconstructing the continuous action vector and then computing the embedding using a linear projection. Discrete tokens are embedded using a dedicated embedding table.

To embed observation tokens $\mathbf{z}$, the tokens of each modality are processed by a modality-specific encoder. The outputs of the encoders are concatenated and further processed by a MLP $g_\psi$ that combines the information into a single vector latent representation.

The image encoder is a convolutional neural network (CNN). Its architecture is given in Table 8.

Categorical variables are embedded using a learned embedding table. For 2D categoricals, shared per-channel embedding tables map tokens to embedding vectors, which are averaged to obtain a single embedding for each multi-channel token vector. For both types of categorical inputs we use 64 dimensional embeddings. The embeddings are concatenated and processed by $g_\psi$.

Table 8: The image observation encoder architecture of the actor-critic controller $\mathcal{C}$.

| Module | Output Shape |
|---|---|
| Input | $256 \times 8 \times 8$ |
| Conv(3, 1, 1) | $128 \times 8 \times 8$ |
| SiLU | $128 \times 8 \times 8$ |
| Conv(3, 1, 1) | $64 \times 8 \times 8$ |
| SiLU | $64 \times 8 \times 8$ |
| Flatten | 4096 |
| Linear | 512 |
| SiLU | 512 |

#### A.5.2 Optimization

$\lambda$-returns are computed for each generated trajectory segment $\hat{\tau} = (\mathbf{z}_1, a_1, \bar{r}_1, d_1, \hat{\mathbf{z}}_2, a_2, \bar{r}_2, d_2, \dots, \hat{\mathbf{z}}_H, a_H, \bar{r}_H, d_H)$:

$$G_t = \begin{cases} \bar{r}_t + \gamma(1 - d_t)((1 - \lambda)\hat{V}_{t+1}^\pi + \lambda G_{t+1}) & t < H \\ \hat{V}_H^\pi & t = H \end{cases},$$

where $\hat{V}^\pi_t = \hat{V}^\pi(\hat{\tau}_{\leq t})$. These $\lambda$-returns are used as targets for critic learning. For policy learning, a REINFORCE [Sutton et al., 1999] objective is used, with a normalized $\hat{V}^\pi$ baseline for variance reduction:

$$\mathcal{L}_\pi(\psi) = \mathop{\mathbb{E}}_{\pi}\left[\sum_{t=1}^{H} \operatorname{sg}\left(\frac{G_t - \hat{V}^\pi_t}{\max(1, c)}\right) \log \pi(a_t|\hat{\tau}_{\leq t-1}, \hat{\mathbf{z}}_t) + w_{\text{ent}} \mathcal{H}(\pi(a_t|\hat{\tau}_{\leq t-1}, \hat{\mathbf{z}}_t))\right],$$

where $c$ is an estimate of the effective return scale similar to DreamerV3 [Hafner et al., 2023] and $w_{\text{ent}}$ is a hyperparameter that controls the entropy regularization weight. $c$ is calculated as the difference between the running average estimators of the $97.5$ and $2.5$ return percentiles, based on a window of return estimates obtained in the last $500$ batches (imagination).

# B Additional Results

The average per-game scores for Atari-100K are presented in Table 9. The performance profile plot for Atari 100K is presented in Figure 7.

Table 9: Mean returns on the 26 games of the Atari 100k benchmark followed by averaged human-normalized performance metrics. Each game score is computed as the average of 5 runs with different seeds. Bold face mark the best score.

| Game | Random | Human | DreamerV3 | TWM | STORM | DIAMOND | REM | Simulus (ours) |
|---|---|---|---|---|---|---|---|---|
| Alien | 227.8 | 7127.7 | 959.4 | 674.6 | **983.6** | 744.1 | 607.2 | 687.2 |
| Amidar | 5.8 | 1719.5 | 139.1 | 121.8 | 204.8 | **225.8** | 95.3 | 102.4 |
| Assault | 222.4 | 742.0 | 705.6 | 682.6 | 801.0 | 1526.4 | 1764.2 | **1822.8** |
| Asterix | 210.0 | 8503.3 | 932.5 | 1116.6 | 1028.0 | **3698.5** | 1637.5 | 1369.1 |
| BankHeist | 14.2 | 753.1 | **648.7** | 466.7 | 641.2 | 19.7 | 19.2 | 347.1 |
| BattleZone | 2360.0 | 37187.5 | 12250.0 | 5068.0 | **13540.0** | 4702.0 | 11826.0 | 13262.0 |
| Boxing | 0.1 | 12.1 | 78.0 | 77.5 | 79.7 | 86.9 | 87.5 | **93.5** |
| Breakout | 1.7 | 30.5 | 31.1 | 20.0 | 15.9 | 132.5 | 90.7 | **148.9** |
| ChopperCommand | 811.0 | 7387.8 | 410.0 | 1697.4 | 1888.0 | 1369.8 | 2561.2 | **3611.6** |
| CrazyClimber | 10780.5 | 35829.4 | 97190.0 | 71820.4 | 66776.0 | **99167.8** | 76547.6 | 93433.2 |
| DemonAttack | 152.1 | 1971.0 | 303.3 | 350.2 | 164.6 | 288.1 | **5738.6** | 4787.6 |
| Freeway | 0.0 | 29.6 | 0.0 | 24.3 | 0.0 | **33.3** | 32.3 | 31.9 |
| Frostbite | 65.2 | 4334.7 | 909.4 | **1475.6** | 1316.0 | 274.1 | 240.5 | 258.4 |
| Gopher | 257.6 | 2412.5 | 3730.0 | 1674.8 | **8239.6** | 5897.9 | 5452.4 | 4363.2 |
| Hero | 1027.0 | 30826.4 | **11160.5** | 7254.0 | 11044.3 | 5621.8 | 6484.8 | 7466.8 |
| Jamesbond | 29.0 | 302.8 | 444.6 | 362.4 | 509.0 | 427.4 | 391.2 | **678.0** |
| Kangaroo | 52.0 | 3035.0 | 4098.3 | 1240.0 | 4208.0 | 5382.2 | 467.6 | **6656.0** |
| Krull | 1598.0 | 2665.5 | 7781.5 | 6349.2 | 8412.6 | **8610.1** | 4017.7 | 6677.3 |
| KungFuMaster | 258.5 | 22736.3 | 21420.0 | 24554.6 | 26182.0 | 18713.6 | 25172.2 | **31705.4** |
| MsPacman | 307.3 | 6951.6 | 1326.9 | 1588.4 | **2673.5** | 1958.2 | 962.5 | 1282.7 |
| Pong | -20.7 | 14.6 | 18.4 | 18.8 | 11.3 | **20.4** | 18.0 | 19.9 |
| PrivateEye | 24.9 | 69571.3 | 881.6 | 86.6 | **7781.0** | 114.3 | 99.6 | 100.0 |
| Qbert | 163.9 | 13455.0 | 3405.1 | 3330.8 | **4522.5** | 4499.3 | 743.0 | 2425.6 |
| RoadRunner | 11.5 | 7845.0 | 15565.0 | 9109.0 | 17564.0 | 20673.2 | 14060.2 | **24471.8** |
| Seaquest | 68.4 | 42054.7 | 618.0 | 774.4 | 525.2 | 551.2 | 1036.7 | **1800.4** |
| UpNDown | 533.4 | 11693.2 | 7567.1 | **15981.7** | 7985.0 | 3856.3 | 3757.6 | 10416.5 |
| #Superhuman (↑) | 0 | N/A | 9 | 8 | 9 | 11 | 12 | **13** |
| Mean (↑) | 0.000 | 1.000 | 1.124 | 0.956 | 1.222 | 1.459 | 1.222 | **1.645** |
| Median (↑) | 0.000 | 1.000 | 0.485 | 0.505 | 0.425 | 0.373 | 0.280 | **0.982** |
| IQM (↑) | 0.000 | 1.000 | 0.487 | 0.459 | 0.561 | 0.641 | 0.673 | **0.990** |
| Optimality Gap (↓) | 1.000 | 0.000 | 0.510 | 0.513 | 0.472 | 0.480 | 0.482 | **0.412** |

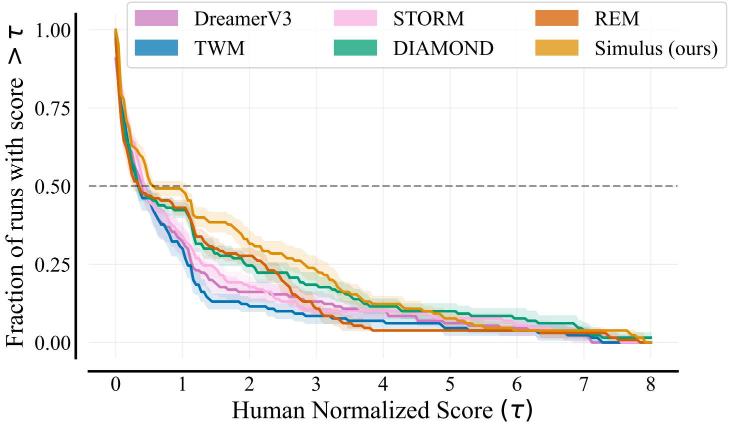


Figure 7: Performance profile. For each human-normalized score value on the x-axis, the curve of each algorithm represents the fraction of runs achieving a score greater than that value. Shaded regions denote pointwise 95% confidence intervals, computed using stratified bootstrap sampling [Agarwal et al., 2021].

### B.1 Interfering Objectives in RSSM Optimization

Here, we study the interplay between the objectives of a Dreamer-like world model, PWM [Wang et al., 2024b], which uses a slightly modified version of the recurrent state space model (RSSM) of Dreamer. Concretely, we aim to understand whether the representation and sequence modeling objectives interfere by decoupling the optimization of the encoder-decoder models from that of the world model. We opted for this implementation due to its simplicity, fast runtime, and accessibility, as it is written in PyTorch [Paszke et al., 2019].

Formally, the model consists of the following components:

$$\begin{aligned} \text{Encoder:} &\quad z_t \sim q_\theta(z_t|o_t), \\ \text{Decoder:} &\quad \hat{o}_t \sim p_\theta(\hat{o}_t|z_t), \\ \text{Sequence model:} &\quad h_t, x_t = f_\theta(x_{t-1}, z_{t-1}, a_{t-1}), \\ \text{Dynamics predictor:} &\quad \hat{z}_t \sim p_\theta(\hat{z}_t|h_t). \end{aligned}$$

We omit the reward and termination predictors and objectives for brevity. Note that in contrast to the RSSM in Dreamer, the encoder and decoder models do not depend on the recurrent state $h_t, x_t$. The optimization objective of PWM is given by

$$\mathcal{L}(\theta) = \mathbb{E}_{q_\theta}[\sum_{t=1}^{T} \beta_{\text{pred}}\mathcal{L}_{\text{pred}}(\theta) + \beta_{\text{dyn}}\mathcal{L}_{\text{dyn}}(\theta) + \beta_{\text{rep}}\mathcal{L}_{\text{rep}}(\theta)],$$

where $\beta_{\text{pred}}, \beta_{\text{dyn}}$, and $\beta_{\text{rep}}$ are coefficients and

$$\begin{aligned} \mathcal{L}_{\text{pred}}(\theta) &= \|\hat{o}_t - o_t\|_2^2, \\ \mathcal{L}_{\text{dyn}}(\theta) &= \max(1, \mathrm{KL}[\mathrm{sg}(q_\theta(z_t|o_t)) \| p_\theta(\hat{z}_t|h_t)]), \\ \mathcal{L}_{\text{rep}}(\theta) &= \max(1, \mathrm{KL}[q_\theta(z_t|o_t) \| \, \mathrm{sg}((p_\theta(\hat{z}_t|h_t))]). \end{aligned}$$

To decouple the optimization, we modify the sequence model by introducing a stop-gradient operator on the encoder's output during world model training:

$$h_t, x_t = f_\theta(x_{t-1}, \mathrm{sg}(z_{t-1}), a_{t-1}).$$

Moreover, this modification allows to train the encoder-decoder models using large batches of single frames, rather than small highly-correlated batches of long trajectories. This further highlights the flexibility and advantage of a modular design.

We compare the original PWM algorithm to its decoupled variant, PWM-decoupled, across four Atari environments: `Breakout`, `DemonAttack`, `Hero`, and `RoadRunner`. These are games where PWM performed either particularly well (`Hero` and `RoadRunner`) or poorly (`Breakout` and `DemonAttack`). Each variant is trained online from scratch on each game. The results are presented in Figure 8. In addition, we present the achieved episodic returns in Figure 9, and the reconstruction quality of example episodes in Figure 10 (PWM) and Figure 11 (PWM-decoupled).

Although our results are based on a single random seed and are limited to only four environments, we observe a consistent trend. First, the reconstruction losses are consistently and significantly lower when decoupling the optimization, while the dynamics losses are significantly higher. This suggests that the objectives are interfering.

Second, we observe similar or better episodic returns (Figure 9) using the decoupled optimization, suggesting that the higher dynamics loss might not lead to worse world modeling performance in practice. Note that a higher dynamics loss in this case does not necessarily mean worse performance, as for example multiple discrete combinations could represent the same or similar frame. Thus, when the dynamics model fails to predict a specific combination, it leads to high loss values while the underlying representations are accurate.

Lastly, we report that similar trends were observed when training only the world model in an offline, supervised-learning fashion on pre-collected datasets. We explored this setting to eliminate complexities that may arise due to the online collection of the data.

While the presented preliminary results are noisy and limited, we believe that they uncover an interesting observation on the design and optimization of current world models.

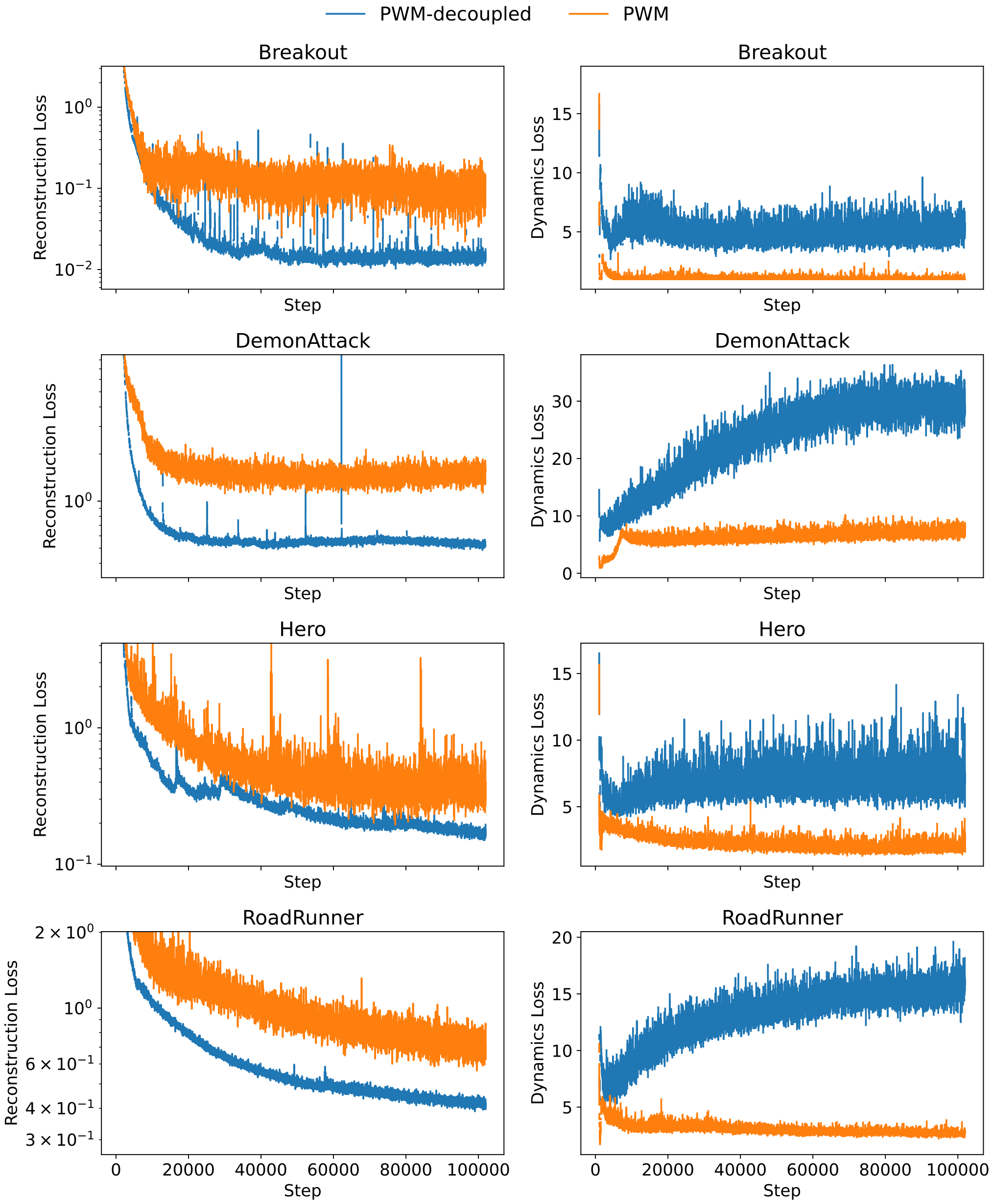


Figure 8: Reconstruction ($\mathcal{L}_{\text{pred}}$) and dynamics ($\mathcal{L}_{\text{dyn}}$) losses of PWM and PWM-decoupled on four Atari games (single seed). The first column uses a log-scaled y-axis. Decoupling the optimization objectives consistently reduces reconstruction loss while increasing dynamics loss, suggesting interference between the two objectives.

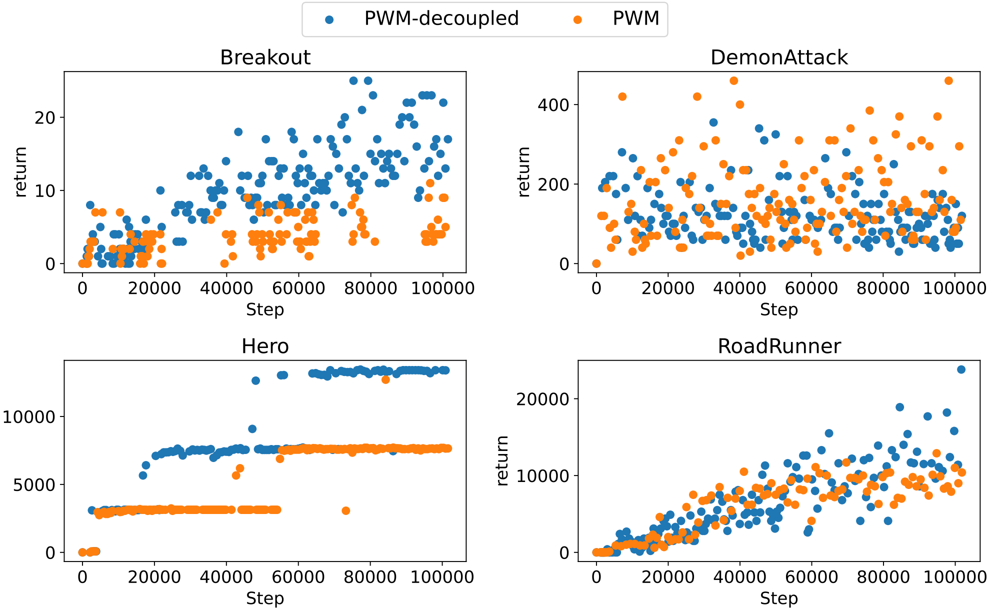


Figure 9: Agent episodic returns throughout training of PWM and PWM-decoupled on four Atari games (single seed). Each marker corresponds to a single episode.

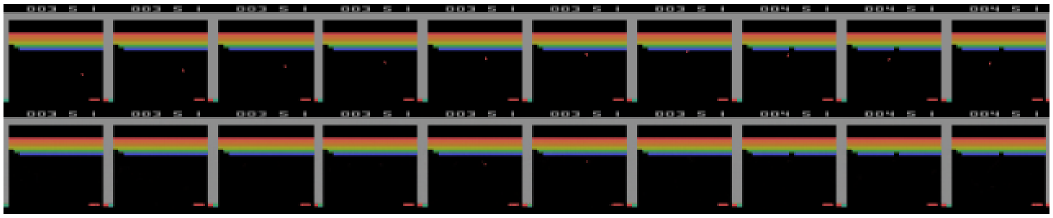


Figure 10: Ground truth (top) and reconstructed (bottom) frames from a training episode of PWM after 50K steps (half way though training). Notably, the ball is missing in most frames, suggesting the reason for its poor performance in this game.

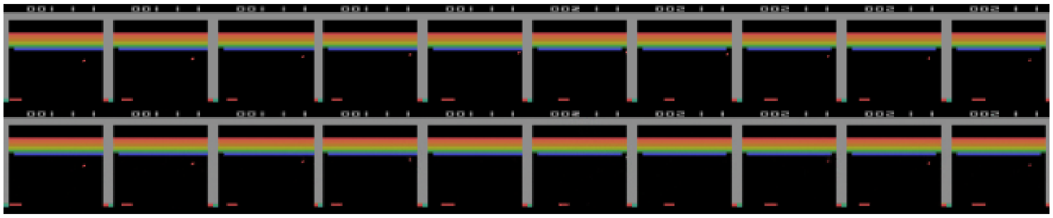


Figure 11: Ground truth (top) and reconstructed (bottom) frames from a training episode of PWM-decoupled after 50K steps (half way though training). Here, the ball is reconstructed in most frames, demonstrating the significantly improved representation performance.

# C Implementation Details

**Ablation Studies** For the Atari 100K benchmark, we conducted ablations on the following games: `Assault`, `Breakout`, `ChopperCommand`, `CrazyClimber`, `JamesBond`, `Kangaroo`, `Seaquest`, and `UpNDown`. For the DeepMind Control Suite, we used the tasks: `acrobot-swingup`, `cartpole-swingup-sparse`, `cheetah-run`, `finger-turn-hard`, `hopper-stand`, `pendulum-swingup`, `reacher-hard`, and `walker-run`.

**Code** We open-source our code and trained model weights. Our code is written in Pytorch [Paszke et al., 2019].

**Hardware** All Atari and DMC experiments were performed on V100 GPUs, while for Craftax a single RTX 4090 was used.

**Run Times** Experiments on Atari require approximately 12 hours on an RTX 4090 GPU and around 29 hours on a V100 GPU. For DMC, the runtime is about 40 hours on a V100 GPU. Craftax runs take roughly 94 hours, equivalent to 3.9 days.

**Craftax** The official environment provides the categorical variables in one-hot encoding format. Our implementation translates these variables to integer values which can be interpreted as tokens.

**Setup in Atari Freeway** For the Freeway environment, we adopted a modified sampling strategy where a temperature of 0.01 is used instead of the standard value of 1, following [Micheli et al., 2023, Cohen et al., 2024]. This adjustment helps directing the agent toward rewarding paths. Note that alternative approaches in the literature tackle the exploration challenge through different mechanisms, including epsilon-greedy exploration schedules and deterministic action selection via argmax policies [Micheli et al., 2023].